

3 Experiments

So far, we have only looked at the methods mathematically.

The calculations tell us what *could* make Bitplanes or Backward Walk faster, but they cannot tell us whether that actually happens on real embeddings.

In this section, I will test the three methods directly.

There are two things I want to understand.

First, I want to compare the search methods under the **same cluster selection**. In other words, if A, B and C are all given exactly the same documents to search, does Bitplanes or Backward Walk actually search those documents faster than simply scanning them?

After that, I will allow each method to choose its own cluster settings and search parameters. This answers the more practical question: if I were actually building the fastest version of each method, which one would be faster at the same recall?

Finally, I will look at cases where the result changes, such as returning one document versus returning 100 documents. This helps us understand not only **which method wins**, but also **when and why**.

3.1 Experiment setup

I use two document collections:

Collection	Embedding model	Documents	Queries	Binary dimensions tested
MS MARCO	Nomic v1.5	1,000,000	1,000	64, 256, 768
Quora	Qwen3 0.6B	522,931	1,000	32, 256, 1024

For each collection, I first generate the document and query embeddings once. The experiments then start from those already-generated vectors.

This is important because the thing I am trying to compare is the **search stage**, not how fast Nomic or Qwen can generate an embedding.

The cached MS MARCO documents and 1,000 selected official queries use Nomic v1.5 [6, 4]. The complete Quora corpus and 1,000 selected official test queries use Qwen3-Embedding-0.6B [7, 5]. Each query selection was fixed before the searches. None of the selected Quora queries has identical text or ID in that corpus.

For one comparison, I keep the following fixed:

- the same documents;
- the same 1,000 queries;
- the same binary dimension;
- the same scoring function;
- the same number of requested results K ;
- the same exact reference answers.

The idea is simple: **A, B and C should solve exactly the same search problem.**

If B is faster than A, I want that difference to come from the search method itself, not because B was given easier queries or a different definition of the correct result.

Changing the binary dimension, for example from 32 bits to 256 bits, is treated as a separate experiment because it also changes the ranking that we are searching for.

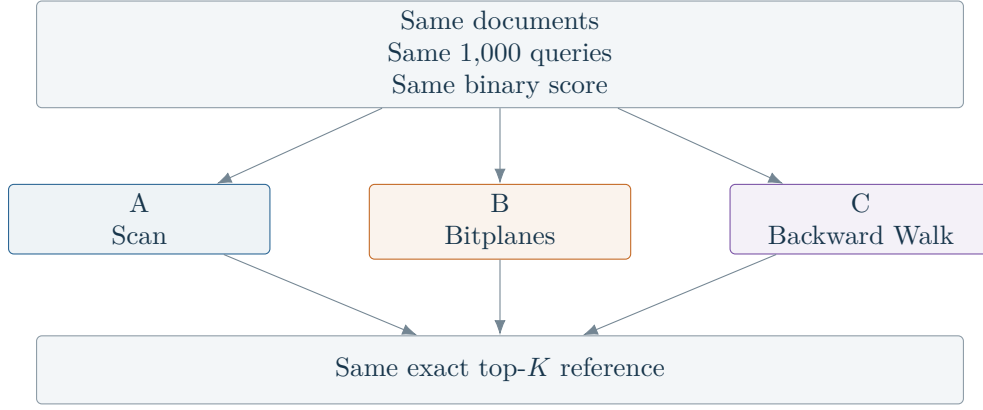


Figure 1: Within one comparison, A, B and C use the same documents, queries, score and reference. The local comparison and whole-collection comparison use different reference sets, explained below.

3.2 What do I measure?

For every query, the timer starts after the query embedding has already been produced.

For the complete-query comparison, I measure:

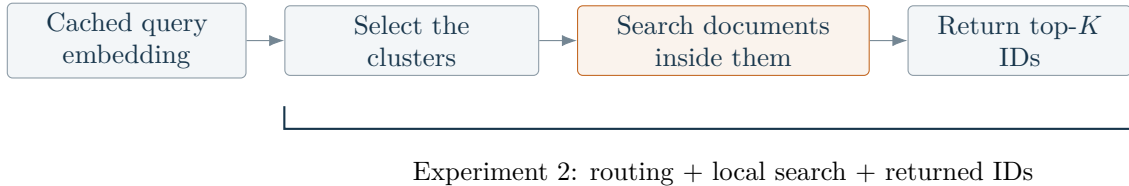


Figure 2: Complete-query time begins with a cached embedding. Experiment 1 reuses cluster selections and times only the local call and returned IDs.

So the complete-query latency includes both:

1. selecting which clusters to open; and
2. searching the documents inside those clusters.

It does not include embedding generation or the later reranking stage.

For the first experiment, I save the selected clusters before timing. Its local-search timer excludes routing. The two sets of timings answer different questions and come from separate runs; I do not subtract one table’s median from another to estimate routing time.

I also record:

- recall;
- number of documents inside the opened clusters;
- number of documents that receive the full score;
- Bitplane word visits for B;
- prefix depths and prefix lookups for C;
- peak process memory.

This lets us answer not only **Which method is faster?** but also **Why is it faster?**

The machine is an Apple M5 Max with 128 GiB physical memory. Each search runs one native query at a time. The complete-query experiments use one Faiss routing thread. The earlier sweep

used a 32 GB process cap; the new local comparison uses a common 64 GB cap. Actual peaks are reported separately, so these limits are not assumed to determine the outcome.

3.3 How do I know whether a result is correct?

Before testing A, B or C, I first calculate the exact binary result for every query.

For one query, I score **every document in the collection** using the same binary score described earlier.

Suppose the exact top five are [12, 7, 91, 4, 30] and one method returns [12, 7, 91, 4, 85].

It recovered four of the five correct IDs, so:

$$\text{Recall@5} = 4/5 = 80\%.$$

I repeat this over all 1,000 queries and report the average recall. Equal scores use document ID to decide the reference order.

So when I say **99% recall** for the whole-collection comparison, I specifically mean: the method recovers 99% of the exact top- K document IDs under the binary score being tested, on average.

It does **not** mean 99% human relevance, and it does not necessarily mean 99% agreement with the original full embedding. I will look at that distinction separately later.

3.4 Experiment 1: give every method the same clusters

The first experiment asks the simplest possible question:

If A, B and C are given the exact same documents, which search method is faster?

Normally, the pipeline first selects some clusters. For this experiment, I freeze the first part.

A, B and C receive exactly the same opened clusters for every query.

So if those clusters contain 50,000 documents:

- A may score all 50,000;
- B may use Bitplanes and score only some of them;
- C may start from a prefix and expose only some of them.

But all three started from the same 50,000 documents.

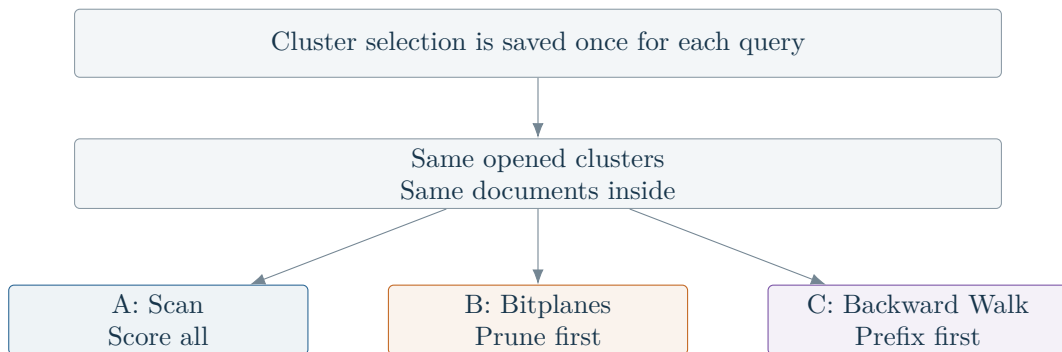


Figure 3: A, B and C search the same documents. Only the local search changes, and routing is outside this experiment's timer.

The reason for doing this experiment is straightforward.

Suppose B is faster than A here. Then we know the improvement is actually coming from the Bitplane search, provided both meet the same local recall requirement.

But suppose B is only faster when it also uses a completely different cluster layout. Then the story is different: the advantage may come from a combination of **cluster selection + Bitplanes**, rather than Bitplanes alone.

Both results are useful, but I want to separate them.

3.4.1 Local recall

In this experiment, I also calculate a second type of recall.

Imagine the selected clusters contain 50,000 documents. I can fully scan those 50,000 documents and find their exact top- K . That becomes the **local reference**.

Then:

- A always has 100% local recall because it scans all 50,000;
- B may lose some local results if it stops branching early;
- C may lose some if it stops at a prefix before reaching them.

This lets us distinguish two different ways of missing a result:

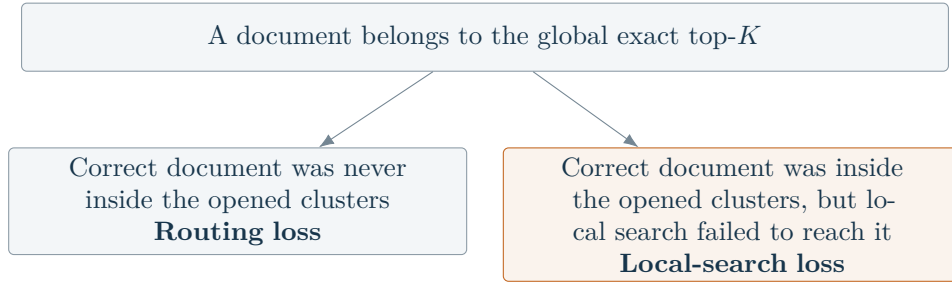


Figure 4: Global recall includes both losses. Local recall compares against a full scan of the already-selected clusters, so it isolates the second source of loss.

The normal global recall still matters for the final system, but local recall makes this particular experiment much easier to understand.

3.4.2 Same routing, different local search

I reused the three routing settings selected by the Qwen32, $K = 1$, 99% complete-query comparison shown later. Each setting was then tested with all three methods.

B used leaf size 128, node budget 8192 and deeper-first ties. C started at depth 4, with target zero and the exact-stop rule. These local settings stayed the same across the three layouts. I repeated each comparison in three fresh processes, rotating method order. The times below measure only local search and returned arrays; cluster selection was saved before timing.

Local search	Rows fully scored	Local recall	Global recall	Local time (ms)
4096 clusters, 180 opened: the same 24,181 rows per query				
A: scan	24,181	100%	99.0%	0.0978
B: Bitplanes	17,936	100%	99.0%	0.1297
C: Backward Walk	15,793	100%	99.0%	0.1004
16 clusters, 9 opened: the same 295,031 rows per query				
A: scan	295,031	100%	99.2%	0.9402
B: Bitplanes	818	100%	99.2%	0.0565
C: Backward Walk	169,362	100%	99.2%	0.4806
1024 clusters, 78 opened: the same 41,147 rows per query				
A: scan	41,147	100%	99.0%	0.1379
B: Bitplanes	4,854	100%	99.0%	0.1099
C: Backward Walk	26,153	100%	99.0%	0.1103

Rows are means over the 1,000 queries; times are medians over all 3,000 repeated observations. Every method in a three-row block received identical cluster IDs and recovered every local top-1 reference in these runs.

The large-cluster result answers the question directly. Scanning the same 295,031 documents took 0.9402 ms, while B took 0.0565 ms and scored only 818. B’s three process medians were 0.0525–0.0609 ms, compared with A’s 0.9301–0.9525 ms. The reduction is therefore a local-search benefit, not merely a different routing choice.

C also helped on those large clusters: it reduced the scored rows to 169,362 and took 0.4806 ms. B was still faster. With the intermediate layout, B and C were close to each other and both faster than A. Their repeat ranges overlap, so the small B/C difference is not a clear winner.

With many small clusters, B’s filtering cost more than it saved. C’s local time was close to A’s, with overlapping repeat ranges. The benefit therefore depends on the selected documents and cluster sizes, even when the query and binary representation stay fixed.

3.4.3 The 256-bit comparison

I also repeated the local comparison with 256-bit codes and $K = 100$, which is closer to the representation size in Exa’s published pipeline. Each collection used A’s selected 99%-global-recall routing: 4096 clusters with 668 probes for Qwen, and 4096 clusters with 1414 probes for Nomic.

The same B and C local settings were used as above:

Collection	Method	Rows scored	Local recall	Local time (ms)
Qwen256	A	86,504	100%	2.3576
	B, leaf 128	86,504	100%	2.7690
	C, start 4	86,504	100%	2.7649
Nomic256	A	345,279	100%	8.9999
	B, leaf 128	104,233	84.14%	4.6308
	C, start 4	345,279	100%	10.5210

Nomic’s B run was faster, but it did not reach 99% local recall. Its global recall was 83.34%, compared with 99.00% for the exact local searches. That is not a valid high-recall speedup.

I also gave B a leaf size large enough to scan every opened cluster immediately, and C its depth-zero scan. This checks that neither method is forced to perform filtering when it cannot help. Among the two local choices tested for each method, the fastest settings reaching 99% local recall were:

Collection	A	B: immediate scan	C: depth-zero scan
Qwen256	2.3576 ms	2.4086 ms	2.5096 ms
Nomic256	8.9999 ms	9.3839 ms	9.8842 ms

All these settings had 100% local recall. They did not establish a pruning advantage over A. This follow-up tested 25 local settings across five fixed layouts, rather than repeating the full parameter sweep. It shows what happened at those settings, not the best possible local configuration.

All 75,000 query records were saved, including the low-recall result. The 60,000 calls in exact modes matched the saved local reference IDs. Input hashes matched before and after the run, and all workers recorded the same power configuration. The largest worker peak was 229.7 MB, well below the common 64 GB cap.

3.5 Experiment 2: let each method choose its own settings

The first experiment intentionally fixes the clusters. But this may not be the best way to actually use B or C.

For example, Bitplanes may work better if I use **larger clusters**. A larger cluster means more documents arrive at the local search stage, which would normally make A slower. But B may be able to cheaply remove most of those documents before fully scoring them.

So for the second experiment, I let each method choose the configuration that works best for it.

For A, I vary:

- number of clusters C ;
- number of opened clusters P .

For B, I vary:

- C ;
- P ;
- leaf size;
- node budget.

For C, I vary:

- C ;
- P ;
- starting prefix depth;
- stopping rule.

All methods can use the same available routing layouts. The short-code follow-up includes clusters trained from either float vectors or binary document signs.

For each target recall, such as 80%, 90%, 95%, 99%, I choose the fastest tested configuration that reaches at least that recall.

This gives us the practical comparison:

If I were actually deploying the fastest tested version of A, B and C, which one would be fastest at the same recall?

I repeat the selected settings in three fresh processes with varied run order. The reported medians combine the repeated query times. This sweep contains 3,679 qualified A/B/C settings from 4,024 scheduled settings, with three grids partial under saved time limits. Every completed setting uses all 1,000 queries. This is a tested parameter grid, not a proof of a global optimum.

3.6 Main result: Qwen 32-bit, return one document

I will first look at Qwen with 32-bit binary documents and $K = 1$, because this is the clearest case where the three methods behave differently.

The fastest qualifying configurations are:

Required recall	A: Scan	B: Bitplanes	C: Backward Walk
80%	0.0308 ms	0.0295 ms	0.0318 ms
90%	0.0498 ms	0.0432 ms	0.0497 ms
95%	0.0805 ms	0.0520 ms	0.0799 ms
99%	0.1948 ms	0.0809 ms	0.1688 ms

At the 99% recall target, B takes 0.0809 ms compared with 0.1948 ms for A. So B is about $2.4\times$ faster in this setting.

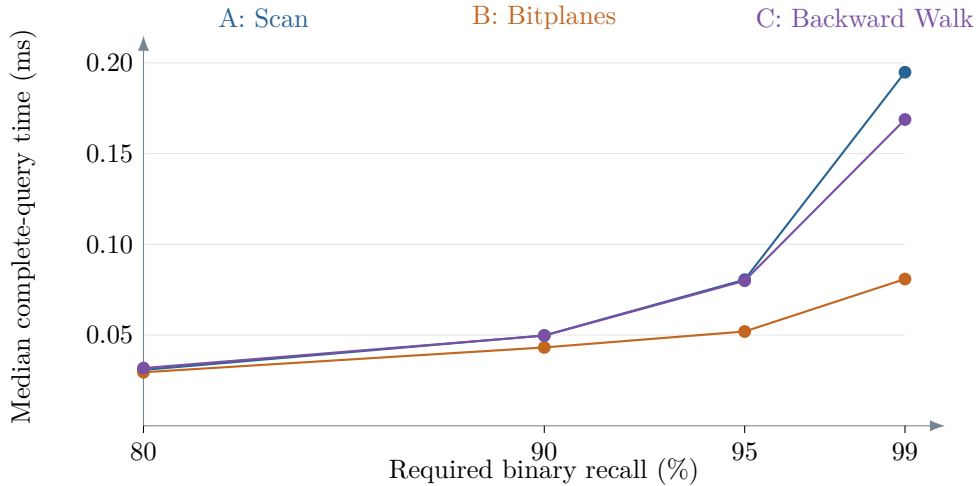


Figure 5: Qwen32, $K = 1$. B has the lowest repeated median at these four recall targets. This figure uses the repeated timings in the table.

3.7 Why is Bitplanes faster here?

The selected 99% configurations are:

Method	Clusters	Opened clusters	Local search
A	4096	180	scan
B	16	9	leaf 128, budget 8192, deeper-first ties
C	1024	78	start depth 4, target zero, exact stop

Now look at how many documents each method actually touches:

Method	Documents inside opened clusters	Documents fully scored
A	24,181	24,181
B	295,031	818
C	41,147	26,153

These document counts are means per query. The achieved global recalls are 99.0%, 99.2% and 99.0% for A, B and C.

This is the most important part of the result.

A tries to make the cluster selection very selective. B does almost the opposite:



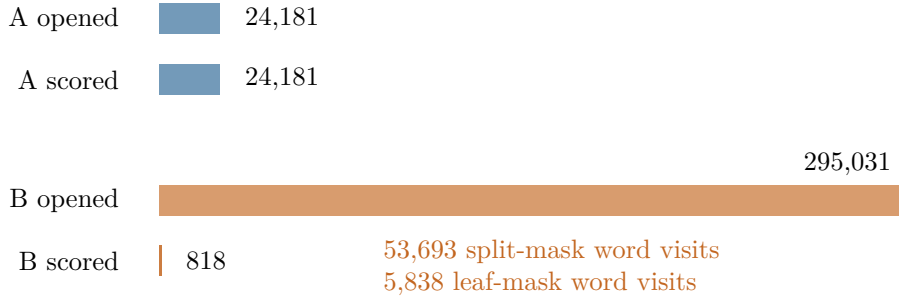
Figure 6: The selected A and B configurations use different routing strategies. B opens more documents, then avoids most full scores through its local search.

So B is not faster because fewer documents reach the opened clusters. It actually opens **more than ten times as many documents as A**.

The difference is that Bitplanes remove almost all of them before the expensive full-score calculation. B scores only around

$$818/295031 \approx 0.28\%$$

of its opened documents.



All bars use the same document-count scale

Figure 7: At the selected 99% setting, B opens about 295,000 documents but scores only 818. Its bitmap-word visits are shown separately because they are different work from full document scores.

This connects directly back to the calculation in Section 2. Bitplanes only make sense if **the full scores we avoid are more expensive than the bitmap work we add**.

In this setting, B avoids roughly 294,000 full scores per query while performing around 59,000 bitmap-word visits. The comparison against scanning B's same opened clusters in Section 3.4 checks that local saving directly. The independently selected complete-query comparison then shows whether B still beats A's own preferred routing.

The counters are not interchangeable units of CPU time. I have not separately timed every score, mask pass and queue operation, so I do not assign invented millisecond costs to those components.

3.8 What happens to Backward Walk?

C also avoids some full scores. It opens around 41,147 documents but only fully scores around 26,153.

It therefore removes roughly 15,000 full scores. However, it also performs prefix searches and range expansion.

At the selected setting, it visits around 3.5 prefix depths and performs around 495 boundary searches per query. Its final time is 0.1688 ms, which is lower than A’s independently selected configuration, but still much slower than B.

Because A and C use different cluster layouts here, this result alone cannot tell us whether the gain comes from Backward Walk itself or from C’s preferred routing configuration. The fixed-cluster results in Section 3.4 separate those questions.

3.9 When does Bitplanes stop helping?

The previous result uses $K = 1$. Now consider the same type of comparison when we ask for $K = 100$.

At 99% recall:

Representation	A	B	C
Nomic 64	1.915 ms	2.071 ms	1.875 ms
Nomic 256	13.453 ms	13.866 ms	14.645 ms
Nomic 768	45.088 ms	45.217 ms	45.196 ms
Qwen 32	0.400 ms	0.439 ms	0.404 ms
Qwen 256	3.564 ms	3.721 ms	4.039 ms
Qwen 1024	22.863 ms	23.098 ms	22.930 ms

The behavior is now very different. At these selected settings:

- B performs no splits;
- C starts at depth zero;
- all three methods score the same documents within each row.

So the pruning advantage disappears. C’s small timing lead on Nomic64 does not come from skipping scores.

When we only need one result, B can search for one very promising region and prune many alternatives. When we need 100 results, more branches may remain relevant. B must keep exploring until enough good documents have been found, which can remove much of its pruning advantage.

In these tested settings, the strongest Bitplane advantage appears when the number of requested results is small enough that many branches can be safely ignored. This is not a guarantee for every model, code length or recall target.

3.10 Binary length is a separate tradeoff

The 32-bit result above is fast, but a 32-bit binary vector does not produce the same ranking as the full embedding.

To measure this difference, I compare the exact binary results with the original full-vector score.

Representation	Binary top-1 matches full-vector best	Full-vector best appears in binary top-100
Qwen 32	14.0%	73.0%
Qwen 256	75.8%	100.0%
Qwen 1024	88.6%	100.0%
Nomic 64	17.6%	65.6%
Nomic 256	48.0%	97.2%
Nomic 768	70.2%	99.9%

This result is not comparing A, B and C. It answers a different question:

How much does the binary representation itself change the ranking?

For example, Qwen32 may be extremely cheap to search, but its exact binary top-1 agrees with the full-vector best on only 14% of the queries. So search speed and representation quality should be treated separately.

Here “matches” means the full-float score is within 10^{-5} of the exhaustive full-float best. Different document IDs can tie. The top-100 column rescues the exact binary candidates; it is not measured end-to-end quality for an approximate A/B/C configuration, and neither column is a human relevance judgment.

3.11 When does Backward Walk work especially well?

Backward Walk is designed to be very cheap when the query’s specific binary prefix already corresponds to stored documents.

I test this directly using Qwen32 with no cluster routing. Among the 1,000 queries, 82 have their complete 32-bit query pattern present in the collection.

Using the same globally selected configuration for each method across all queries:

Query group	Queries	A (μ s)	B (μ s)	C (μ s)
All queries	1000	2481.7	124.1	1163.5
Exact 32-bit code exists	82	2481.3	128.5	4.9
Exact code absent	918	2481.7	123.8	1170.7

When the exact code exists, C finds a tiny range immediately and scores only around 1.7 documents on average. Its search therefore looks approximately like:



Figure 8: For the 82 exact-code queries, C reaches a small range immediately and stops after scoring it. The same configuration is used for the other 918 queries.

In this case, Backward Walk is extremely fast. However, this only happens for 82 of the 1,000 queries.

For the other 918 queries, C must progressively relax the prefix and scores about 303,177 documents on average. B remains faster over the complete 1,000-query batch.

The groups were defined by existing code membership, not favorable timings, and their settings were not retuned. Every method returns the exact binary best result here. C starts at depth 32 with target zero and the exact-stop bound; B uses leaf 2048 and an unlimited budget.

The observed advantage is specifically for a complete 32-bit match. An occupied shorter prefix alone does not prove that C can stop exactly; the score bound must still hold. Matching binary codes also do not imply identical text.

3.12 Memory

All selected configurations from the earlier complete-query sweep remain far below its 32 GB process limit.

For the Qwen32 $K = 1$ 99% settings:

Method	Index + router fields	Measured process peak
A	8.924 MB	77.971 MB
B	10.463 MB	72.139 MB
C	10.598 MB	77.529 MB

The largest selected process peak across that completed sweep is around 505.6 MB. MB means one million bytes. Field totals exclude allocator capacity, overlapping buffers and other runtime memory.

So at the collection sizes tested here, memory does not determine the winner. The main difference between the methods is query-time work. The local follow-up records its own process peaks under the common 64 GB cap; these are separate from the older numbers in this table.

3.13 Additional checks

I also verify that the work counters match the calculations from Section 2.

For Bitplanes, I record:

- split-mask words;
- leaf-mask words;
- number of visited branches;
- rows fully scored.

For Backward Walk, I record:

- starting depth;
- final depth;
- number of prefix depths visited;
- number of boundary searches;
- rows fully scored.

The stored index-size formulas are also checked against the actual index fields. Across the earlier study, 890 index-storage checks and 3,849,076 query-count checks matched their formulas.

For Backward Walk’s exact stopping rule, the predicted stopping depth and number of scored rows match all 3,000 checked query records: the same 1,000 global Qwen32 queries at starting depths 1, 4 and 32. The exact reference score is used only for this offline check. Actual search uses the scores it has found.

The separate probability study retained all three seeds. On its fixed Qwen32 layout, probabilities 0, 0.1 and 0.2 gave top-1 recalls of 87.30%, 83.33% and 70.90% at budget 128. At budget 8192 they all reached 95.30%, while median time rose from 0.0766 to 0.0857 and 0.0959 ms. Nomic64 also showed no consistent time gain at matching recall. This did not improve the tested tradeoff; it does not prove randomness can never help.

These checks do not determine which method is fastest; the measured query time does that. Their purpose is to verify that the explanation of the work matches what the implementation actually performs.

3.14 Experiment summary

The experiments show that there is no single method that is best in every setting.

For Qwen32 with $K = 1$, Bitplanes were substantially faster in the independently selected comparison. At the 99% binary recall target, B took 0.0809 ms compared with 0.1948 ms for A.

The reason is not that B opened fewer documents. B actually opened a much larger set of documents, but its query-dependent Bitplane traversal reduced roughly 295,000 opened documents to only around 818 full scores.

When K increased to 100 at the same 99% target, the selected configurations across the six representations became scan-like. Backward Walk showed a more specific advantage when the complete query code existed; otherwise it often needed substantial prefix relaxation.

The fixed-cluster comparison above separates local-search behavior from routing. Its conclusions are limited to the tested local settings, while the complete-query sweep allows each method to choose its own routing as well.

3.15 Code and reproduction

The local comparison is in `experiments/same_clusters.py`. It saves cluster selections and local references first, then runs each method in fresh processes. Returned IDs, work counts, local/global recall and timing records are under `results/same-clusters-2026-09-13/`.

The earlier complete-query sweep is under `results/prefix-study-2026-09-13/`. Its README links the final tables to the source settings and repeated observations. Data preparation is described in `docs/expanded-pool-preparation.md`. All searches use the common C++ scorer in `cpp/score.hpp`; the report diagrams illustrate those operations rather than adding different algorithms.

4 Limitations and next steps

The timings describe this CPU implementation. All three methods reuse the same C++ lookup-table scorer, which adds entries using float64. Exa’s production system and its SIMD kernels are not reproduced here. A faster scoring kernel can change the balance between scoring documents and maintaining an index search.

I hold the collection, model and code length fixed when choosing each method’s parameters. Shorter codes change the reference ranking, so their binary recall does not establish the same retrieval quality as a longer representation. The source datasets provide human relevance labels for that separate evaluation.

These are static indexes built before querying. The experiment does not measure document updates or concurrent requests. In particular, keeping rows sorted still costs work when new documents are added. The memory formulas count the binary search index and its query buffers; an application that keeps uncompressed vectors or document text in RAM must add that storage.

Prefix expansion also appears in LSH Forest [8] and PUFFINN [9]. Those methods use multiple hash indexes and different stopping rules. Their recall guarantees do not transfer to a single prefix in the embedding’s original coordinate order.

5 Conclusion

For Qwen32 with one returned result, B takes 0.081 ms at 99.2% binary recall, versus A’s 0.195 ms and C’s 0.169 ms at 99.0%. All methods can choose binary-trained clusters. With 100 results at the 99% target, the selected B and C settings instead become scans. C has a narrower exact-search benefit: it takes 4.875 μ s on the 82 queries whose complete code exists, while B remains faster over all 1,000 queries.

These results identify useful tested settings, not a universal optimum. Qwen32’s exact binary top 1 agrees with the full-float best score on only 14% of queries, so the short-code speed does not preserve full-embedding quality. For a fixed representation and recall requirement, the decision remains whether the scores avoided pay for the added mask or prefix work. The measured counts let us check that explanation directly.

References

- [1] The Exa Team. *How we built a web-scale vector database*. 17 December 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] Y. Malkov and D. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2016; revised 2018. <https://arxiv.org/abs/1603.09320>.
- [3] A. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS, 2022. <https://arxiv.org/abs/2205.13147>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model card, accessed 13 September 2026. The saved manifest identifies the exact model revision. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Y. Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. 2025. <https://arxiv.org/abs/2506.05176>. Encoding recipe: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [6] P. Bajaj et al. *MS MARCO: A Human Generated Machine Reading Comprehension Dataset*. 2016; revised 2018. <https://arxiv.org/abs/1611.09268>. Passage-ranking data: <https://github.com/microsoft/MSMARCO-Passage-Ranking>.
- [7] N. Thakur et al. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets and Benchmarks, 2021. <https://arxiv.org/abs/2104.08663>. Dataset catalog: <https://github.com/beir-cellar/beir>.
- [8] M. Bawa, T. Condie and P. Ganesan. *LSH Forest: Self-Tuning Indexes for Similarity Search*. WWW, 2005, pp. 651–660. <https://www.cs.princeton.edu/courses/archive/spring06/cos592/bib/LSHForest-bawa05.pdf>.
- [9] M. Aumüller, T. Christiani, R. Pagh and M. Vesterli. *PUFFINN: Parameterless and Universally Fast Finding of Nearest Neighbors*. ESA, 2019. <https://doi.org/10.4230/LIPIcs.ESA.2019.10>.